

# Workshop 2: Intro to pandas

TECH2: Introduction to Programming, Data, and Information Technology

Richard Foltyn

*NHH Norwegian School of Economics*

October 9, 2026

See GitHub repository for notebooks and data:

<https://github.com/richardfoltyn/TECH2-H26>

## Contents

|   |                                                 |    |
|---|-------------------------------------------------|----|
| 1 | Exercise 1: Data cleaning                       | 1  |
| 2 | Exercise 2: Selecting subsets of data           | 4  |
| 3 | Exercise 3: Labor market statistics for the US  | 7  |
| 4 | Exercise 4: Daily stock-market returns          | 11 |
| 5 | Exercise 5: Working with string data (advanced) | 13 |

## Exercise 1: Data cleaning

Before doing actual data analysis, we usually first need to clean the data. This might involve steps such as dealing with missing values and encoding categorical variables as integers. In this exercise, you will perform such steps based on the Titanic passenger data.

1. Load the Titanic data set in `titanic.csv` located in the `data/` folder.
2. Report the number of observations with missing Age, for example using `isna()`.
3. Compute the average age in the data set. Use the following approaches and compare your results:

1. Use pandas's `mean()` method.

*Hint:* Select the Age column first, then call its `mean()` method.

2. Convert the Age column to a NumPy array using `to_numpy()`. Experiment with NumPy's `np.mean()` and `np.nanmean()` to see if you obtain the same results.

4. Replace all missing ages with the mean age you computed above, rounded to the nearest integer. You can either identify the missing values using a boolean mask and assign the mean age to those entries, or use `fillna()`.

Convert this updated Age column to integer type using `astype()`.

*Hint:* Use `astype(int)`.

5. Generate a new column `Female` which takes on the value one if `Sex` is equal to "female" and zero if `Sex` is equal to "male". This is called an *indicator* or *dummy* variable, and is useful for numerical analysis or regression. Delete the original column `Sex`.

*Hint:* Create a new column using `df['new_column'] = values`, replacing `new_column` with the desired column name and `values` with the data to store.

6. Save your cleaned data set as `titanic-clean.csv` using `to_csv()` with `,` as the field separator. Tell `to_csv()` to *not* write the `DataFrame` index to the CSV file as it's not needed in this example.

---

### Solution.

#### Part 1 — Loading the data

```
[1]: # Import the pandas library
import pandas as pd

# Relative path to data directory
DATA_PATH = '../data'

# Path to Titanic CSV file
file = f'{DATA_PATH}/titanic.csv'

# Read in the CSV file, use default separator (comma)
df = pd.read_csv(file)
```

#### Part 2 — Number of missing values

We can count the number of missing values directly by summing the return values of `isna()`, which returns `True` whenever an observation is missing:

```
[2]: # Number of missing age observations
df['Age'].isna().sum()
```

```
[2]: np.int64(177)
```

Alternatively, the number of non-missing values can be displayed using the `info()` method. For larger `DataFrame` objects, pandas does not automatically report the number of non-missing ("Non-Null") observations, so we might need to request this explicitly by passing the `show_counts=True` argument.

```
[3]: # Display nonmissing counts for each column
df.info(show_counts=True)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null    int64
1   Survived     891 non-null    int64
2   Pclass       891 non-null    int64
3   Name         891 non-null    str
4   Sex          891 non-null    str
5   Age          714 non-null    float64
6   Ticket       891 non-null    str
7   Fare         891 non-null    float64
8   Cabin        204 non-null    str
9   Embarked     889 non-null    str
dtypes: float64(2), int64(3), str(5)
memory usage: 69.7 KB
```

### Part 3 — Compute mean age

We compute the mean age using the three different methods. As you can see, `np.mean()` cannot deal with missing values and returns NaN (“not a number”).

```
[4]: import numpy as np

# Compute mean age using the Series.mean() method
mean_age = df['Age'].mean()

# Convert Age column to NumPy array
age_array = df['Age'].to_numpy()

# Compute mean using np.mean()
mean_age_np = np.mean(age_array)

# Compute mean using np.nanmean()
mean_age_np_nan = np.nanmean(age_array)

print(f'Mean age using pandas:      {mean_age:.3f}')
print(f'Mean age using np.mean():   {mean_age_np:.3f}')
print(f'Mean age using np.nanmean(): {mean_age_np_nan:.3f}')

Mean age using pandas:      29.699
Mean age using np.mean():   nan
Mean age using np.nanmean(): 29.699
```

### Part 4 — Replace missing ages and convert to integers

There are several ways to replace missing values. First, we can “manually” identify these using Boolean indexing and assign a new value to such observations.

```
[5]: # Round average age
mean_age = np.round(mean_age)

# Boolean arrays to select missing observations
is_missing = df['Age'].isna()

# Update missing observations with rounded mean age
df.loc[is_missing, 'Age'] = mean_age
```

There is also the convenience routine `fillna()`, which automates this step. To illustrate, we need to reload the original data as we have just replaced all missing values.

```
[6]: # Re-load data to get the original missing values
df = pd.read_csv(file)

df['Age'] = df['Age'].fillna(value=mean_age)
```

We store age as an integer for this exercise.

```
[7]: # Drop fractional parts in the recorded ages.
df['Age'] = df['Age'].astype(int)
```

### Part 5 — Generate Female indicator

An indicator variable can be obtained as a result of a logical operation (`==`, `!=`, etc.). The resulting Series contains True or False values, which we can convert to 1 or 0 by changing the data type to integer.

There are no missing values in Sex here. If there were, this comparison could turn missing values into False, coding them as zero. The commented alternative below uses `map()` to convert 'female' directly

to 1 and 'male' to 0, preserving missing values as NaN. When missing values are present, the result uses floating-point numbers (1.0 and 0.0), since ordinary integer columns cannot store NaN. Values not listed in the mapping also become NaN.

```
[8]: # Generate Boolean array (True/False) whether passenger is female
is_female = df['Sex'] == 'female'

# Add Female dummy variable, converted to integer
df['Female'] = is_female.astype(int)

# Alternative: map Sex directly to 1/0 while preserving missing values
# df['Female'] = df['Sex'].map({'female': 1, 'male': 0})

# Delete original Sex column, no longer needed
del df['Sex']

# Alternatively, you can use
# df = df.drop(columns=['Sex'])
```

## Part 6 — Save cleaned file

We can use `info()` again to confirm that Age has no missing values and all columns are of the desired data type:

```
[9]: df.info(show_counts=True)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  891 non-null   int64
1   Survived     891 non-null   int64
2   Pclass       891 non-null   int64
3   Name         891 non-null   str
4   Age          891 non-null   int64
5   Ticket       891 non-null   str
6   Fare         891 non-null   float64
7   Cabin        204 non-null   str
8   Embarked     889 non-null   str
9   Female       891 non-null   int64
dtypes: float64(1), int64(5), str(4)
memory usage: 69.7 KB
```

```
[10]: # Save cleaned file
file_clean = f'{DATA_PATH}/titanic-clean.csv'

df.to_csv(file_clean, sep=',', index=False)
```

---

## Exercise 2: Selecting subsets of data

In this exercise, you are asked to select subsets of macroeconomic data for the United States based on some criteria.

1. Load the annual data from FRED which are located in `FRED_annual.xlsx` in the `data/FRED` folder. Print the list of columns and the number of non-missing observations.

Since we are dealing with time series data, set the column Year as the DataFrame index.

2. Print all observations for the 1960s decade using at least two different methods.
3. Using the data in the column GDP, compute the annual GDP growth in percent and store it in the column GDP\_growth. Select the years in which
  1. GDP growth was above 5%.
  2. GDP growth was negative, but inflation was still above 5% (such episodes are called “stagflation” since usually negative GDP growth is associated with low inflation).

Use at least two methods for the stagflation selection.

*Hint:* You can compute changes relative to the previous observation using the `pct_change()` method.

---

### Solution.

#### Part 1 — Load and prepare data

```
[11]: import pandas as pd

# Relative path to data directory
DATA_PATH = '../..data'

# Path to annual FRED data in Excel format
file = f'{DATA_PATH}/FRED/FRED_annual.xlsx'

# Read in data
df = pd.read_excel(file)
```

We print the columns and their corresponding number of observations using the `info()` method. This data set is small enough so that pandas automatically reports the number of observations, but we can still pass `show_counts=True` if we want.

```
[12]: # List columns, force pandas to show number of observations
df.info(show_counts=True)
```

```
<class 'pandas.DataFrame'>
RangeIndex: 72 entries, 0 to 71
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Year        72 non-null    int64
1   GDP         72 non-null    float64
2   CPI         72 non-null    float64
3   UNRATE      72 non-null    float64
4   FEDFUNDS    72 non-null    float64
5   INFLATION   71 non-null    float64
dtypes: float64(5), int64(1)
memory usage: 3.5 KB
```

In this case, the result is the same if you call `info()` without additional arguments, but that might not be the case for larger DataFrame objects, depending on your local configuration settings:

```
[13]: df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 72 entries, 0 to 71
Data columns (total 6 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Year        72 non-null    int64
1   GDP         72 non-null    float64
2   CPI         72 non-null    float64
3   UNRATE      72 non-null    float64
4   FEDFUNDS    72 non-null    float64
5   INFLATION   71 non-null    float64
dtypes: float64(5), int64(1)
memory usage: 3.5 KB

```

We set Year as the DataFrame index.

```

[14]: # Set Year as index, discard previous (default) index
df = df.set_index('Year')

```

## Part 2 — Select data from the 1960s

There are various ways to select all observations for the 1960s. Since we have set the Year as an index, we can use `.loc[]` to select by label:

```

[15]: df.loc[1960:1969]

```

```

[15]:
      GDP  CPI  UNRATE  FEDFUNDS  INFLATION
Year
1960  3500.3  29.6    5.5      3.2    1.369863
1961  3590.1  29.9    6.7      2.0    1.013514
1962  3810.1  30.3    5.6      2.7    1.337793
1963  3976.1  30.6    5.6      3.2    0.990099
1964  4205.3  31.0    5.2      3.5    1.307190
1965  4478.6  31.5    4.5      4.1    1.612903
1966  4773.9  32.5    3.8      5.1    3.174603
1967  4904.9  33.4    3.8      4.2    2.769231
1968  5145.9  34.8    3.6      5.7    4.191617
1969  5306.6  36.7    3.5      8.2    5.459770

```

Alternatively, we can use the `query()` method, which allows us to select observations not only based on column values, but also based on values of the index (Year in this case):

```

[16]: df.query('Year >= 1960 & Year <= 1969')

```

```

[16]:
      GDP  CPI  UNRATE  FEDFUNDS  INFLATION
Year
1960  3500.3  29.6    5.5      3.2    1.369863
1961  3590.1  29.9    6.7      2.0    1.013514
1962  3810.1  30.3    5.6      2.7    1.337793
1963  3976.1  30.6    5.6      3.2    0.990099
1964  4205.3  31.0    5.2      3.5    1.307190
1965  4478.6  31.5    4.5      4.1    1.612903
1966  4773.9  32.5    3.8      5.1    3.174603
1967  4904.9  33.4    3.8      4.2    2.769231
1968  5145.9  34.8    3.6      5.7    4.191617
1969  5306.6  36.7    3.5      8.2    5.459770

```

## Part 3 — GDP growth

We can use the `pct_change()` method to compute changes relative to the previous year. To convert these changes to percent, we need to multiply the values returned by this method by 100.

Alternatively, we can calculate growth directly as  $100 \times (GDP_t - GDP_{t-1}) / GDP_{t-1}$ . We use `shift()` with an argument of 1 to align each year's GDP with the previous year's value. The first growth rate is missing because there is no previous observation.

```
[17]: # Compute GDP growth as percentage change in GDP vs. the previous year
df['GDP_growth'] = df['GDP'].pct_change() * 100

# Alternative: calculate growth manually using the previous year's GDP
gdp_previous = df['GDP'].shift(1)

# Growth (%) = (current GDP - previous GDP) / previous GDP * 100
gdp_growth_manual = (df['GDP'] - gdp_previous) / gdp_previous * 100
```

We can now use this column to select episodes when GDP growth was above 5%, for example by using the `query()` method.

```
[18]: # Select high growth episodes using query()
df.query('GDP_growth > 5')
```

```
[18]:
```

|      | GDP     | CPI   | UNRATE | FEDFUNDS | INFLATION | GDP_growth |
|------|---------|-------|--------|----------|-----------|------------|
| Year |         |       |        |          |           |            |
| 1955 | 3083.0  | 26.8  | 4.4    | 1.8      | -0.371747 | 7.134170   |
| 1959 | 3412.4  | 29.2  | 5.4    | 3.3      | 1.038062  | 6.931562   |
| 1962 | 3810.1  | 30.3  | 5.6    | 2.7      | 1.337793  | 6.127963   |
| 1964 | 4205.3  | 31.0  | 5.2    | 3.5      | 1.307190  | 5.764443   |
| 1965 | 4478.6  | 31.5  | 4.5    | 4.1      | 1.612903  | 6.498942   |
| 1966 | 4773.9  | 32.5  | 3.8    | 5.1      | 3.174603  | 6.593578   |
| 1972 | 5780.0  | 41.8  | 5.6    | 4.4      | 3.209877  | 5.255490   |
| 1973 | 6106.4  | 44.4  | 4.9    | 8.7      | 6.220096  | 5.647059   |
| 1976 | 6387.4  | 56.9  | 7.7    | 5.0      | 5.762082  | 5.386989   |
| 1978 | 7052.7  | 65.2  | 6.1    | 7.9      | 7.590759  | 5.535105   |
| 1984 | 8195.3  | 103.9 | 7.5    | 10.2     | 4.317269  | 7.236042   |
| 2021 | 21532.4 | 271.0 | 5.4    | 0.1      | 4.673619  | 6.151988   |

Next, we want to select stagflationary periods when the US economy stagnated (GDP growth below 0%), but inflation was nevertheless high (above 5%). We can achieve this by using `query()` with a Boolean *and* operator, `&`:

```
[19]: df.query('GDP_growth < 0 & INFLATION > 5')
```

```
[19]:
```

|      | GDP    | CPI  | UNRATE | FEDFUNDS | INFLATION | GDP_growth |
|------|--------|------|--------|----------|-----------|------------|
| Year |        |      |        |          |           |            |
| 1974 | 6073.4 | 49.3 | 5.6    | 10.5     | 11.036036 | -0.540417  |
| 1975 | 6060.9 | 53.8 | 8.5    | 5.8      | 9.127789  | -0.205816  |
| 1980 | 7257.3 | 82.4 | 7.2    | 13.4     | 13.498623 | -0.257009  |
| 1982 | 7307.3 | 96.5 | 9.7    | 12.3     | 6.160616  | -1.803400  |

Alternatively, we can combine two individual Boolean Series, one for each condition, and use the result to select the relevant years.

```
[20]: condition = (df['GDP_growth'] < 0) & (df['INFLATION'] > 5)
df[condition]
```

```
[20]:
```

|      | GDP    | CPI  | UNRATE | FEDFUNDS | INFLATION | GDP_growth |
|------|--------|------|--------|----------|-----------|------------|
| Year |        |      |        |          |           |            |
| 1974 | 6073.4 | 49.3 | 5.6    | 10.5     | 11.036036 | -0.540417  |
| 1975 | 6060.9 | 53.8 | 8.5    | 5.8      | 9.127789  | -0.205816  |
| 1980 | 7257.3 | 82.4 | 7.2    | 13.4     | 13.498623 | -0.257009  |
| 1982 | 7307.3 | 96.5 | 9.7    | 12.3     | 6.160616  | -1.803400  |

## Exercise 3: Labor market statistics for the US

In this exercise, you are asked to compute some descriptive statistics for the unemployment rate and the labor force participation rate for the United States. The latter is the percentage of the civilian noninstitutional population aged 16 and older who are in the labor force (employed or unemployed). Both rates are expressed as percentages.

1. Load the monthly time series from FRED which are located in `FRED_monthly.csv` in the `data/FRED` folder.

*Hint:* Use `pd.read_csv(..., parse_dates=['DATE'])` to automatically parse strings stored in the `DATE` column as dates.

Since we are dealing with time series data, set the column `DATE` as the `DataFrame` index. Then print the list of columns and the number of non-missing observations.

2. For the columns `UNRATE` (unemployment rate) and `LFPART` (labor force participation rate), compute and report the mean, minimum, and maximum values for the whole sample. Round your results to one decimal place.

*Hint:* You can use the `DataFrame` methods `mean()`, `min()`, and `max()` to compute the desired statistics.

*Hint:* You can use the `DataFrame` method `round()` to round the results to one decimal place.

3. You are interested in how the average unemployment rate evolved over the last few decades.
  - Add a new column `Decade` to the `DataFrame` which contains the starting year for each decade (e.g., this value should be 1950 for the years 1950-1959, and so on).

*Hint:* The decade can be computed from the column `Year` using truncated integer division:

```
df['Year'] // 10 * 10
```

  - Write a loop to compute and report the average unemployment rate (column `UNRATE`) for each decade.

Include only the decades from 1950 to 2010 for which you have all observations.

---

### Solution.

#### Part 1 — Load and inspect the data

When reading CSV files, we can use the `parse_dates` argument to tell pandas which columns should be interpreted as dates.

```
[21]: import numpy as np
import pandas as pd

# Relative path to data directory
DATA_PATH = '../data'

# Path to monthly FRED CSV file
file = f'{DATA_PATH}/FRED/FRED_monthly.csv'

# Read the CSV file, parsing the 'DATE' column as dates
df = pd.read_csv(file, parse_dates=['DATE'])
```

Since we are dealing with time series data, we use `DATE` as the index:

```
[22]: # Set the column DATE as the index, discarding the previous (default) index
df = df.set_index('DATE')
```

We print the columns and their corresponding number of non-missing observations using `info()`. For larger DataFrame objects, pandas does not automatically report the number of non-missing (“Non-Null”) observations, so we might need to request this explicitly by passing the `show_counts=True` argument.

As you can see, not all variables have the same number of observations. For example, the real interest rate (REALRATE) and the Federal Funds Rate (FEDFUNDS) are only available for the later years.

```
[23]: # Report columns and number of non-missing observations
df.info(show_counts=True)
```

```
<class 'pandas.DataFrame'>
DatetimeIndex: 936 entries, 1948-01-01 to 2025-12-01
Data columns (total 7 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   Year        936 non-null    int64
 1   Month       936 non-null    int64
 2   CPI         935 non-null    float64
 3   UNRATE      935 non-null    float64
 4   FEDFUNDS    858 non-null    float64
 5   REALRATE    528 non-null    float64
 6   LFPART      935 non-null    float64
dtypes: float64(5), int64(2)
memory usage: 58.5 KB
```

## Part 2 — Whole-sample labor market statistics

There are various ways to solve this part. One approach is to select the columns UNRATE and LFPART and compute the desired statistics (mean, min, max), and round the result to one decimal place:

```
[24]: print('Average:')
print(df[['UNRATE', 'LFPART']].mean().round(1))
```

```
Average:
UNRATE      5.7
LFPART     62.8
dtype: float64
```

```
[25]: print('Minimum:')
print(df[['UNRATE', 'LFPART']].min().round(1))
```

```
Minimum:
UNRATE      2.5
LFPART     58.1
dtype: float64
```

```
[26]: print('Maximum:')
print(df[['UNRATE', 'LFPART']].max().round(1))
```

```
Maximum:
UNRATE     14.8
LFPART     67.3
dtype: float64
```

Alternatively, we can perform this task in a single line leveraging the `describe()` method. Recall that `describe()` computes various summary statistics for each column:

```
[27]: df[['UNRATE', 'LFPART']].describe()
```

```
[27]:
```

|       | UNRATE     | LFPART     |
|-------|------------|------------|
| count | 935.000000 | 935.000000 |
| mean  | 5.666524   | 62.832086  |

|     |           |           |
|-----|-----------|-----------|
| std | 1.706049  | 2.885587  |
| min | 2.500000  | 58.100000 |
| 25% | 4.300000  | 59.800000 |
| 50% | 5.500000  | 62.900000 |
| 75% | 6.700000  | 65.900000 |
| max | 14.800000 | 67.300000 |

The output contains statistics we are not interested in, but we can use `.loc[]` to select only the desired rows and round them to one decimal place:

```
[28]: df[['UNRATE', 'LFPART']].describe().loc[['mean', 'min', 'max']].round(1)
```

```
[28]:      UNRATE  LFPART
mean      5.7    62.8
min       2.5    58.1
max      14.8    67.3
```

### Part 3 — Labor market statistics by decade

First, we add a new column `Decade` which contains the starting year of each decade corresponding to the observation date:

```
[29]: df['Decade'] = df['Year'] // 10 * 10
```

The following table shows that not all decades include the full set of observations:

```
[30]: df['Decade'].value_counts().sort_index()
```

```
[30]: Decade
1940      24
1950     120
1960     120
1970     120
1980     120
1990     120
2000     120
2010     120
2020      72
Name: count, dtype: int64
```

We therefore restrict our attention to the decades from 1950 to 2010 and compute the average unemployment rate for each decade:

```
[31]: decades = np.arange(1950, 2011, 10)

print('Unemployment rate by decade:')
for decade in decades:
    mean = df.loc[df['Decade'] == decade, 'UNRATE'].mean()
    print(f'Decade starting in {decade}: {mean:5.1f}')
```

```
Unemployment rate by decade:
Decade starting in 1950:  4.5
Decade starting in 1960:  4.8
Decade starting in 1970:  6.2
Decade starting in 1980:  7.3
Decade starting in 1990:  5.8
Decade starting in 2000:  5.5
Decade starting in 2010:  6.2
```

Note that this task can be achieved much more elegantly using grouping operations, which we will study in the next lectures.

## Exercise 4: Daily stock-market returns

In this exercise, you will compute daily returns for the S&P 500 index and count trading days with losses or large gains. The column `SP500` contains the index values.

1. Load `SP500.csv` from the `data/stockmarket` folder, parse `Date` as dates, and set it as the `DataFrame` index. Restrict the data to observations after January 1, 2020.

*Hint:* Use `pd.read_csv(..., parse_dates=['Date'])` to automatically parse the strings in the `Date` column as dates.

2. Compute daily returns in percent using `pct_change()` and store them in a new column `Return`. Explain why the first return is missing.

*Hint:* Multiply the result of `pct_change()` by 100 to express returns in percent.

3. Report the number and percentage of trading days with losses, i.e., negative returns. Exclude days with missing returns from the denominator.

*Hint:* Summing a Boolean Series counts the number of True values. Use `count()` to count nonmissing observations.

4. Report the number and percentage of trading days with gains of more than 5%.

5. Report the number and percentage of days with negative returns on both the current and the previous trading day. A run of three loss days contributes two qualifying days. For the percentage, include only days with both returns observed.

*Hint:* Use `shift()` to obtain the previous trading day's return, and combine the two conditions using `&`.

---

### Solution.

#### Part 1 — Load and restrict the data

We parse `Date` as dates and use it as the index:

```
[32]: import pandas as pd

# Relative path to data directory
DATA_PATH = '../data'

file = f'{DATA_PATH}/stockmarket/SP500.csv'
df_sp500 = pd.read_csv(file, parse_dates=['Date'])
df_sp500 = df_sp500.set_index('Date')
```

We then restrict the data to observations after January 1, 2020:

```
[33]: df_sp500 = df_sp500.loc[df_sp500.index > '2020-01-01']
```

#### Part 2 — Compute daily returns

We multiply the fractional changes returned by `pct_change()` by 100 to obtain returns in percent. The first return is missing because the previous trading day is outside the restricted sample.

```
[34]: df_sp500['Return'] = df_sp500['SP500'].pct_change() * 100
df_sp500.head()
```

```
[34]:
```

|      | SP500 | Return |
|------|-------|--------|
| Date |       |        |

|            |        |           |
|------------|--------|-----------|
| 2020-01-02 | 3257.9 | NaN       |
| 2020-01-03 | 3234.9 | -0.705976 |
| 2020-01-06 | 3246.3 | 0.352407  |
| 2020-01-07 | 3237.2 | -0.280319 |
| 2020-01-08 | 3253.1 | 0.491165  |

### Part 3 — Count days with losses

We first generate the Series `losses` which is True for each negative return. Summing these values counts the days with losses. We divide by the number of nonmissing returns to obtain the fraction of trading days with losses. The `.2%` format displays this fraction as a percentage with two decimal places.

```
[35]: # Generate Boolean array (True/False) whether the return is negative
losses = df_sp500['Return'] < 0
# Count the number of non-missing returns (days with trading data)
n_days = df_sp500['Return'].count()

print(f'Days with losses: {losses.sum()}')
print(f'Percentage of trading days: {losses.sum() / n_days:.2%}')
```

```
Days with losses: 685
Percentage of trading days: 45.48%
```

### Part 4 — Count large gains

Since `Return` is expressed in percent, we compare it with 5. We use the same number of nonmissing returns to compute the percentage of trading days:

```
[36]: large_gains = df_sp500['Return'] > 5

print(f'Days with gains of more than 5%: {large_gains.sum()}')
print(f'Percentage of trading days: {large_gains.sum() / n_days:.2%}')
```

```
Days with gains of more than 5%: 7
Percentage of trading days: 0.46%
```

### Part 5 — Count consecutive loss days

We use `shift()` to obtain the previous trading day's return and require both returns to be negative. Here, shifting by one row refers to the previous trading day, which need not be the previous calendar day.

For the percentage, we include only days with both returns observed in the denominator. The `notna()` method identifies nonmissing values.

```
[37]: # Shift return values by 1
prev_return = df_sp500['Return'].shift()
# Generate Boolean array (True/False) whether the return is negative on consecutive days
consecutive_losses = losses & (prev_return < 0)
# Identify valid observations (non-missing returns on both days)
valid = df_sp500['Return'].notna() & prev_return.notna()

print(f'Days with losses on consecutive trading days: {consecutive_losses.sum()}')
print(f'Percentage of trading days: {consecutive_losses.sum() / valid.sum():.2%}')
```

```
Days with losses on consecutive trading days: 296
Percentage of trading days: 19.67%
```

## Exercise 5: Working with string data (advanced)

Most of the data we deal with contain strings, i.e., text data (names, addresses, etc.). Often, such data is not in the format needed for analysis, and we have to perform additional string manipulation to extract the exact data we need. This can be achieved using the pandas [string methods](#).

To illustrate, we use the Titanic data set for this exercise.

1. Load the Titanic data and restrict the sample to men. (This simplifies the task. Some married women's entries contain both their husband's name and their maiden name.)
2. Print the first five observations of the Name column. As you can see, the data is stored in the format "Last name, Title First name" where the title is something like Mr., Rev., etc.
3. Split the Name column by , to extract the last name and the remainder as separate columns. You can achieve this using the `partition()` string method.
4. Split the remainder (containing the title and first name) using the space character " " as separator to obtain individual columns for the title and the first name.

*Hint:* The remainder starts with a space after the comma. Use `.str.strip()` to remove leading and trailing whitespace before splitting it.

5. Store the three data series in the original DataFrame (using the column names `FirstName`, `LastName`, and `Title`) and delete the `Name` column, which is no longer needed.
6. Finally, extract the ship deck from the values in `Cabin`. The ship deck is the first character in the string stored in `Cabin` (A, B, C, ...). You extract the first character using the `get()` string method. Store the result in the column `Deck`.

*Hint:* pandas's string methods can be accessed using the `.str` attribute. For example, to partition values in the column `Name`, you need to use

```
df['Name'].str.partition()
```

---

### Solution.

#### Part 1 — Import data and restrict to male sub-sample

```
[38]: # Path to data directory
DATA_PATH = '../..data'

# Alternatively, load data directly from GitHub
# DATA_PATH = 'https://raw.githubusercontent.com/richardfoltyn/TECH2-H26/main/data'
```

```
[39]: import pandas as pd

# Path to Titanic CSV file
file = f'{DATA_PATH}/titanic.csv'

df = pd.read_csv(file)
```

We restrict the sample either with Boolean indexing or with the `query()` method.

```
[40]: # Restrict sample to men
df = df.loc[df['Sex'] == 'male'].copy()

# Alternatively, we can do this with a query()
# df = df.query('Sex == "male"')
```

## Part 2 — Inspect the Name column

```
[41]: # Print first 5 Name observations
df['Name'].head(5)
```

```
[41]: 0      Braund, Mr. Owen Harris
4      Allen, Mr. William Henry
5      Moran, Mr. James
6      McCarthy, Mr. Timothy J
7      Palsson, Master Gosta Leonard
Name: Name, dtype: str
```

## Part 3 — Split into last name and remainder

Note that `partition()` returns *three* columns, the second one containing the separator you specified. This second column can be ignored.

```
[42]: # Split names by comma, create DataFrame with a column for each token
names = df['Name'].str.partition(',')

# Print first 5 rows of resulting DataFrame
names.head(5)
```

```
[42]:   0  1  2
0  Braund , Mr. Owen Harris
4  Allen , Mr. William Henry
5  Moran , Mr. James
6  McCarthy , Mr. Timothy J
7  Palsson , Master Gosta Leonard
```

```
[43]: # Extract last name stored in 1st column, strip any remaining white space
last_name = names[0].str.strip()

# Print first 5 observations
last_name.head(5)
```

```
[43]: 0      Braund
4      Allen
5      Moran
6      McCarthy
7      Palsson
Name: 0, dtype: str
```

## Part 4 — Split title and first name

```
[44]: # Title and first name (potentially multiple) are separated by space
title_first = names[2].str.strip().str.partition(' ')

title_first.head(5)
```

```
[44]:   0  1  2
0  Mr. Owen Harris
4  Mr. William Henry
5  Mr. James
6  Mr. Timothy J
7  Master Gosta Leonard
```

```
[45]: # Extract title from 1st column, strip any remaining white space
title = title_first[0].str.strip()
title.head(5)
```

```
[45]: 0      Mr.
4      Mr.
5      Mr.
6      Mr.
7    Master
Name: 0, dtype: str
```

```
[46]: # Extract first name(s) from 3rd column, strip any remaining white space
first_name = title_first[2].str.strip()

# Print first 5 observations
first_name.head(5)
```

```
[46]: 0      Owen Harris
4    William Henry
5           James
6      Timothy J
7    Gosta Leonard
Name: 2, dtype: str
```

## Part 5 — Store name components in original DataFrame

```
[47]: # Merge all name components back into original DataFrame
df['FirstName'] = first_name
df['LastName'] = last_name
df['Title'] = title

# Delete Name column
del df['Name']
```

```
[48]: df.head(5)
```

```
[48]: PassengerId  Survived  Pclass   Sex   Age  Ticket     Fare  Cabin \
0           1         0        3  male  22.0  A/5  21171    7.2500  NaN
4           5         0        3  male  35.0  373450    8.0500  NaN
5           6         0        3  male   NaN  330877    8.4583  NaN
6           7         0        1  male  54.0   17463   51.8625  E46
7           8         0        3  male   2.0  349909   21.0750  NaN

   Embarked  FirstName  LastName  Title
0         S    Owen Harris   Braund   Mr.
4         S  William Henry     Allen   Mr.
5         Q         James     Moran   Mr.
6         S    Timothy J  McCarthy   Mr.
7         S   Gosta Leonard   Palsson  Master
```

## Part 6 — Extract deck

We can use the `get()` string method to extract the first element of the cabin string (if present). Note that observations with a missing value for Cabin will also be assigned a missing value for Deck.

```
[49]: df['Deck'] = df['Cabin'].str.strip().str.get(0)
```

```
[50]: # Count passengers with known cabin information by deck
df['Deck'].value_counts().sort_index()
```

```
[50]: Deck
      A    14
      B    20
      C    32
      D    15
      E    17
      F     8
      T     1
      Name: count, dtype: int64
```

---